

Documentação Técnica do Sistema - Varzea Pro

1. Visão Geral do Sistema

O **Varzea Pro** é um sistema web desenvolvido para a gestão de campeonatos de futebol amador (futebol de várzea). O objetivo principal é informatizar e centralizar o controle de equipes, escalação de atletas e o agendamento de partidas, mitigando erros comuns como escalação irregular e conflitos de agendamento.

O sistema foi concebido como uma aplicação monolítica, utilizando o padrão arquitetural MVC (Model-View-Controller).

2. Arquitetura e Tecnologias

O projeto foi construído utilizando as seguintes tecnologias e frameworks:

- **Linguagem:** Java 17
- **Framework Principal:** Spring Boot (v4.0.6)
- **Segurança:** Spring Security
- **Camada de Visão (Frontend):** Thymeleaf (Server-side rendering), HTML5, CSS3, e Bootstrap 5 para responsividade.
- **Persistência de Dados:** Spring Data JPA / Hibernate
- **Banco de Dados:** H2 Database (Rodando em memória para fins acadêmicos e de prototipação).

3. Padrão de Projeto (MVC)

A aplicação está dividida estritamente nas seguintes camadas:

- **Model (Entidades):** Representam as tabelas do banco de dados e as regras de mapeamento ORM (Object-Relational Mapping).
- **Repository:** Interfaces do Spring Data JPA responsáveis pela comunicação direta com o banco H2 (operações CRUD).
- **Service:** Camada intermediária que concentra as regras de negócio (ex: validação de senhas, criptografia, bloqueio de duplicações).
- **Controller:** Responsável por interceptar as requisições HTTP do navegador, chamar os Services adequados e devolver a página HTML (View) correta.

4. Dicionário de Dados (Entidades Principais)

4.1. Entidade: Usuario (Administradores)

Responsável pelo controle de acesso ao painel gerencial.

- id (Long): Chave primária.
- nomeCompleto (String): Nome real do administrador.
- username (String): Credencial de acesso (Login).
- senha (String): Senha criptografada via BCrypt.

- role (String): Nível de permissão (ex: ROLE_ADMIN).
- email (String): Correio eletrônico para contato.

4.2. Entidade: Time

Representa as equipes cadastradas no campeonato.

- id (Long): Chave primária.
- nome (String): Nome da equipe.
- *Relacionamento*: @OneToMany com Jogador (Uma equipe possui vários jogadores).

4.3. Entidade: Jogador

Representa os atletas amadores.

- id (Long): Chave primária.
- nome (String): Nome completo do atleta.
- posicao (String): Posição de atuação (Goleiro, Zagueiro, Lateral, Meio Campo, Atacante).
- *Relacionamento*: @ManyToOne com Time (Vários jogadores pertencem a um time).

4.4. Entidades: Campeonato e Partida

- **Campeonato**: Agrupa os times e o regulamento.
- **Partida**: Registra o confronto. Possui relacionamentos com a entidade Time para definir mandante e visitante, além da data e do campeonato associado.

5. Regras de Negócio e Segurança

1. **Autenticação Obrigatória**: Nenhuma rota do sistema (exceto /login, /registro e console do banco H2) pode ser acessada sem um token de sessão válido. O Spring Security intercepta e bloqueia requisições não autorizadas.
2. **Criptografia Unidirecional**: As senhas dos usuários nunca são salvas em texto puro. O sistema utiliza o algoritmo BCryptPasswordEncoder no momento do registro. Durante o login, o Spring Security realiza a validação do hash.
3. **Integridade Referencial**: O sistema impede a criação de jogadores sem um time associado (validação via formulário e integridade de chave estrangeira no banco).

6. Mapeamento de Rotas (Endpoints da Interface)

- GET / e GET /login: Controle de acesso e autenticação.
- GET e POST /registro: Criação de novos administradores.
- GET /times/listar: Visualização do grid de equipes.
- POST /times/salvar: Persistência de uma nova equipe.

- GET /jogadores/listar: Visualização dos atletas.
- POST /jogadores/salvar: Vinculação e persistência de um jogador a um time.
- *(Idem para rotas de Campeonatos e Partidas)*